

# Supabase RLS, on one page

Row-Level Security done right — the rules, the footguns, the policies, and the test that proves it.

**The one rule that matters.** Every table is reachable from the browser through the `anon` key. RLS is the *only* thing between a user and everyone else's rows. **RLS off = the table is public.** RLS on with no policy = denies everything. Get this right and the leak can't happen.

## 01 The 5 golden rules

1. **Enable RLS on every table** — in the same migration that creates it. `alter table X enable row level security;` No exceptions, even for "internal" tables.
2. **Scope every row to its owner** — store `user_id` (or `workspace_id` for teams) and compare it to `auth.uid()` in every policy.
3. **USING for reads, WITH CHECK for writes.** `USING` filters what they can see/affect; `WITH CHECK` validates what they may write. No `WITH CHECK` = they write rows they can't read.
4. **Never ship `USING (true)`** — that's "everyone, always," the permissive policy that quietly makes a table public. Need public read? Do it on purpose and scope the columns.
5. **The `service_role` key bypasses RLS — keep it server-only.** Never in the browser, never in `NEXT_PUBLIC_*`. Trusted server code only (webhooks, cron).

## 02 The footguns — what actually leaks in prod

- New table shipped with RLS **off** → public. (The #1 Supabase leak.)
- `USING (true)` left in from a tutorial → public read.
- INSERT policy with no `WITH CHECK` → a user inserts rows for another tenant.
- `service_role` key exposed to the client → RLS is irrelevant.
- Trusting the app/ORM `where` instead of RLS → one missing filter is a breach.

## 03 The correct pattern — owner-scoped

```
-- every table, always
alter table notes enable row level security;

create policy "owner reads own notes" on notes for select
  using (user_id = (select auth.uid()));

create policy "owner inserts own notes" on notes for insert
  with check (user_id = (select auth.uid()));

create policy "owner updates own notes" on notes for update
```

```
using (user_id = (select auth.uid()))  
with check (user_id = (select auth.uid()));
```

Wrapping `(select auth.uid())` in a subselect lets Postgres cache it per-statement — a real speed win at scale.

## 04 Multi-tenant (teams / workspaces)

```
-- resolve membership once, keep policies simple + fast  
create or replace function public.user_workspace_ids()  
  returns setof uuid language sql security definer stable as $$  
  select workspace_id from workspace_members where user_id = auth.uid()  
$$;  
  
create policy "members read workspace rows" on projects for select  
  using (workspace_id in (select public.user_workspace_ids()));
```

## 05 Prove it — don't hope

A policy you didn't test is a policy you don't have. The only proof that holds: **sign in as tenant A, try to read tenant B's row, assert you get nothing.**

- ✓ tenant A reads A's rows
- ✓ tenant A gets **0 rows** for B (blocked, not just absent)
- ✓ tenant A **cannot INSERT** a row for B (no WITH CHECK hole)
- ✗ any table without RLS → fail the build

Run it in CI on every migration. A new table without RLS should turn the build red — before it reaches prod.

## 06 30-second checklist

- ☐ RLS enabled on **every** table
- ☐ Every policy scoped to `auth.uid()` / membership
- ☐ `WITH CHECK` on every INSERT / UPDATE

- ☐ No `USING (true)` anywhere
- ☐ `service_role` key server-only
- ☐ An isolation test that fails the build

Make it the default, free: `npx supabase-saas-kit new` · `npx airlock-rls`

**shipsealed.com** — ship Supabase apps that don't leak 🐧